Comprehensive Guide to Alert Brokers for Supernova Science

In the context of the Vera Rubin Observatory (LSST) and current surveys

Table of Contents

1. The Vera Rubin Observatory Alert System
2. What are Alert Brokers and What Are They For?
3. Major Brokers
    - 3.1 ALeRCE
    - 3.2 AMPEL
    - 3.3 ANTARES
    - 3.4 Fink
    - 3.5 Lasair
    - 3.6 Pitt-Google Broker
4. Specialized Brokers (Downstream)
    - 4.1 SNAPS (Solar System)
    - 4.2 POI Broker (Variable Stars)
5. Basic Resources and Tutorials
6. Practical Examples of Filters for Supernovae

---

1. The Vera Rubin Observatory Alert System

The heart of the **Vera C. Rubin Observatory** is the **Legacy Survey of Space and Time (LSST)**. Every night, the telescope will generate a stream of approximately **10 million alerts** for transient and variable objects. These alerts are generated when the image difference software (which compares a new image with a reference one) detects a new light source that was not present before. This stream includes:

- **Supernovae (SNe):** The main target of this guide.
- **Variable Stars:** From Cepheids to cataclysmic variables.
- **Asteroids:** Moving objects within our Solar System.
- **AGN and Active Galactic Nuclei:** Variations in the centers of galaxies.
- **Contaminants:** Artifacts, cosmic rays, reduction errors.

The sheer volume of data is such that no single scientist or research group could analyze it manually. This is where brokers come into play.

2. What are Alert Brokers and What Are They For?

An **alert broker** is a sophisticated software system that sits between the telescope and the scientist. Its main functions are:

- **Ingestion and Storage:** It efficiently receives, processes, and stores the massive stream of alerts in real-time.
- **Enrichment (Cross-matching):** It adds crucial information to each alert by comparing its coordinates with existing astronomical catalogs (e.g., *Gaia* for stars, *SDSS* or *Pan-STARRS* for host galaxies). Knowing whether an event is in a galaxy or a star is fundamental for classification.
- **Classification:** It uses machine learning algorithms (like neural networks, random forests) to assign a probability to each alert of belonging to a certain class (e.g., "SN Ia", "SN II", "variable", "asteroid").
- **Filtering:** It allows users to define custom filters (in Python, SQL, etc.) to receive only the alerts that match their scientific criteria (e.g., "Type Ia supernova candidates with magnitude < 20").

In summary, a broker acts as an **automated research assistant** that sifts through millions of signals to offer you exactly what you are interested in.

---

3. Major Brokers

Here is an in-depth overview of the most promising brokers for supernova science in the LSST era.

3.1 ALeRCE — Automatic Learning for the Rapid Classification of Events

- **Description:** Managed by a Chilean consortium, ALeRCE (which is a tree native to Chile and Argentina) specializes in using state-of-the-art machine learning algorithms (CNN, RNN, etc.) for real-time classification. It currently processes data from the Zwicky Transient Facility (ZTF) and is ready for LSST. Its strength lies in the speed and accuracy of its automatic classification.
- **Resources and Tutorials:**
    - **Official Site:** https://alerce.science/
    - **GitHub (Python Client):** https://github.com/alerce-science/alerce-client
    - **Interactive Tutorials (Jupyter Notebooks):** They offer numerous examples on how to use the client and API to query the database, obtain light curves, and classifications. They are a gold mine for learning (Example here).

- **Usage Example (Python):**
  This script searches for objects classified as supernovae with high probability and prints their coordinates.

```
python
from alerce.core import Alerce
import pandas as pd

# Initialize the client
client = Alerce()

# Search for objects with classification 'SN' and probability > 0.9
# Note: exact parameters may vary; consult the documentation.
results = client.query_objects(
    classifier='rf', # Random Forest classifier
    class_name='SN',
    probability=0.9,
    limit=10
)

# Convert results to a pandas DataFrame for easy viewing
df = pd.DataFrame(results)
if not df.empty:
    print(df[['oid', 'ra', 'dec', 'probability']])
else:
    print("No supernovae found with these criteria.")
```

3.2 AMPEL — Alert Management, Photometry & Evaluation of Light Curves

- **Description:** AMPEL stands out for its modular and scalable architecture. It is designed as a "channel-based" platform where users (or "user groups") can implement their own analysis code. This offers unparalleled flexibility: you are not limited to what the broker provides, but you can run your own algorithms directly on the data.
- **Resources and Tutorials:**
  - **Official Documentation:** https://ampelproject.github.io/
  - **GitHub (ampel-contrib-sample):** https://github.com/AmpelProject/ampel-contrib-sample Contains examples of user code (contrib) for creating your own filters and analyses.

- **Conceptual Filter Example (Python - within an AMPEL channel):**
  The user defines a class that inherits from BaseAlertConsumer and implements the logic.

```
python
# Simplified example of an AMPEL analysis module
from ampel.alert.BaseAlertConsumer import BaseAlertConsumer
from ampel.content.DataPoint import DataPoint

class MySNFilter(BaseAlertConsumer):
    def process(self, alert):
        # Extract photometry and classification (if present)
        phot = alert['prv_candidates'] # previous photometry
        classif = alert.get('classifications', {})

        # Filter for bright, well-classified supernovae
        if classif.get('SN_prob', 0) > 0.85 and alert['magpsf'] <
20.0:
            # If the filter passes, the alert is sent to my
personal channel
            return self._create_t2_alert(alert)
        return None
```

3.3 ANTARES — Arizona–NOIRLab Temporal Analysis & Response to Events System

- **Description:** Developed by a team from the University of Arizona and NOIRLab, ANTARES is a "real-time" broker that emphasizes data enrichment and a Python-based rule filtering system. It is designed to respond quickly to events, ideal for spectroscopic follow-up campaigns.
- **Resources and Tutorials:**
  - **Site and Documentation:** https://antares.noirlab.edu/
  - **Python Client:** https://github.com/astro-umar/antares-client (Note: verify the most up-to-date official repository).
  - **Installation and Usage Tutorials:** Often included in the documentation, showing how to connect to the server, define streams, and receive alerts.
- **Filter Example (Python):**
  In ANTARES, you write a Python function that is applied to each alert. If the function returns True, the alert passes the filter.

```python
# Example filter function for ANTARES
def supernova_filter(alert):
    # 1. Check if it's considered a real event (not a bogus)
    if alert['attributes']['realbogus'] < 0.9:
        return False

    # 2. Check the class provided by ANTARES (e.g., via a classifier)
    # The structure of alert['classifications'] depends on the ANTARES version
    classifications = alert.get('classifications', {})
    if classifications.get('supernova_prob', 0) < 0.7:
        return False

    # 3. (Optional) Filter by magnitude
    if alert['photometry'][-1]['magnitude'] > 20.5:
        return False

    # If all checks pass, the alert is interesting
    return True
```

3.4 Fink

- **Description:** Fink is an open-source broker, guided by an international community (led by French and Australia). Its goal is to maximize the scientific return of LSST by promoting transparency and community participation. It provides not only classifications but also science modules developed by users (e.g., for searching for kilonovae or gravitational lenses).
- **Resources and Tutorials:**
    - **Documentation (ReadTheDocs):** https://fink-broker.readthedocs.io/
    - **Portal and API:** https://fink-broker.org/ , https://ztf.fink-portal.org Rubin LSST soon!
    - Tutorials: The documentation includes numerous Jupyter notebooks on how to query data, use science modules, and even contribute your own module. for ZTF: https://github.com/astrolabsoftware/fink-tutorials and https://doc.ztf.fink-broker.org
- **Usage Example (Python - via REST API):**
  Query the latest alerts classified as supernovae.

```python
import requests
import pandas as pd

# Fink API endpoint
url = "https://fink-portal.org/api/v1/explorer"
params = {
    "class": "Supernova candidate", # Class of interest
    "n": "50"                       # Number of results
}

r = requests.post(url, json=params)

if r.status_code == 200:
    # The data is in JSON format, convertible to a DataFrame
    df = pd.read_json(r.json())
    print(df[['i:objectId', 'i:ra', 'i:dec',
'v:classification']].head())
else:
    print(f"Request error: {r.status_code}")
```

There are different API urls fro ZTF and LSST. We also have different endpoints depending if you want photometry, conesearch, etc More here https://doc.ztf.fink-broker.org/en/latest/services/search/objectid/ for ZTF

3.5 Lasair

- **Description:** The UK's broker, Lasair, is designed to be interactive and easy to use for the astronomical community. Its main features are the ability to run **SQL queries** directly on the alert database, create **interactive filters** via a web interface, and use **watchlists** (lists of known objects, such as host galaxies) to annotate alerts in real-time.
- **Resources and Tutorials:**
    - **Site and Documentation:** https://lasair-ztf.lsst.ac.uk/ (currently for ZTF, but the architecture is for LSST)
    - **Python Client (**lasair**):** https://pypi.org/project/lasair/
    - **Guides and Tutorials:** The official documentation contains numerous guides on how to create filters, use watchlists, and run queries.

- **Filter Example (SQL):**
  This SQL filter looks for objects with a high probability of being supernovae and with a g-band magnitude brighter than 21.

```
sql
-- SQL filter for Lasair
SELECT
    objects.objectId,
    objects.ra,
    objects.dec,
    objects.classification
FROM objects
-- 'crossmatch_tns' is a table that might contain classifications
LEFT JOIN crossmatch_tns ON objects.objectId =
crossmatch_tns.objectId
WHERE
    objects.classification LIKE '%SN%' AND
    objects.mag_g < 21 AND
    objects.probability > 0.8
ORDER BY objects.mag_g ASC
LIMIT 100;
```

Updata on Lasair (25/02/202) -- Lasair system is now open for you to investigate the alerts from the LSST survey of the Rubin Observatory.

For information about the new system, see the documentation (link below). The Lasair Blog (link below) will be the main means of communication from now on. To get a notification of a new post, click the bell symbol on the right and select "Watching". We welcome contributions to the Lasair Blog -- its a fine way to advertise your latest paper. Some recent posts are:

- Fat Kafka Streams

- Handling your Filtered Alerts

- Lasair-LSST for users of Lasair-ZTF

Some useful links for the Lasair/LSST:

Website : https://lasair.lsst.ac.uk

Documentation : https://lasair-lsst.readthedocs.io/

Support : https://community.lsst.org/c/support/support-lasair/55

Lasair blog : https://community.lsst.org/c/sci/lasair-blog/67

API endpoint : https://api.lasair.lsst.ac.uk/api

Kafka : lasair-lsst-kafka.lsst.ac.uk:9092

3.6 Pitt-Google Broker

- **Description:** Developed by the University of Pittsburgh and Google, this broker is an example of a cloud-native architecture, built on Google Cloud Platform (GCP) using services like Pub/Sub for alert streaming and BigQuery for large-scale storage and analysis. It is designed to be extremely scalable and reliable.
- **Resources and Tutorials:**
  - **Official Documentation:** https://mwvgroup.github.io/pittgoogle-client/listings.html
  - **Python client:** pittgoogle-client
  - **API Reference:** Explains how to configure a Google Cloud Platform project. Includes examples on how to access data from BigQuery or subscribe to Pub/Sub topics. Supports integration of user-defined alert filters.
  - **Tutorials**: The pittgoogle-user-demos repo contains tutorials for accessing astronomical data that is served by the Pitt-Google Alert Broker through Google Cloud. The data is available both from within Google Cloud and outside of it. These tutorials demonstrate both.

- **Query Example (on BigQuery):**

```
sql
-- Standard SQL query to find recent, bright supernova candidates
in BigQuery
SELECT
    objectId,
    ra,
    dec,
    mjd,
    magpsf,
    filtername,
    ndethist
FROM
    `bigquery-public-data.pitt_google_broker.alerts`
WHERE
    -- Filter for supernova candidates (the exact column name needs
verification)
    -- and for magnitude < 20
    candidate.magpsf < 20.0
    AND candidate.ndethist ≥ 2 -- At least 2 nights of detection
    AND DATE(candidate.mjd, 1858-11-17) > DATE_SUB(CURRENT_DATE(),
INTERVAL 1 DAY) -- Last day
ORDER BY
    mjd DESC
LIMIT 100;
```

Once the data is in BigQuery, you can use SQL for complex analyses.

---

4. Specialized Brokers (Downstream)

In addition to the "general" brokers, there are brokers that focus on specific scientific areas. They are called "downstream" because they rely on the alert streams already processed by the main brokers.

4.1 SNAPS — Solar System Notification Alert Processing System

- **Description:** Dedicated exclusively to Solar System objects. SNAPS receives alerts, identifies those showing apparent motion (asteroids), and calculates their orbits. It is essential for separating asteroids from supernovae and for the study of the Solar System.

4.2 POI Broker — Point of Interest

- **Description:** This broker focuses on the light curves of variable stars and objects of particular interest. Instead of analyzing all alerts, it starts from a predefined list of "points of interest" (e.g., known stars, galaxies with historical SNe) and analyzes their light curves in detail, providing classifications and physical parameters.

---

5. Basic Resources and Tutorials to Get Started

To work with brokers, familiarity with some tools is essential:

- **Key Languages/Tools:**
  - **Python 3.x:** The standard language of astronomy.
  - **Broker Clients:** Each broker provides a Python client to interact with its APIs.
  - **Pandas & NumPy:** For data manipulation and table analysis.
  - **Astropy:** For handling coordinates, times (MJD), and astronomical units.
  - **SQL/BigQuery:** Essential for brokers like Lasair and Pitt-Google that offer SQL queries on data.
  - **Kafka/PubSub:** For those who want to process the stream in real-time (more advanced).
- **Direct Links to Tutorials:**
  - **ALeRCE Notebooks:** Look for the "Tutorial" or "Learn" section on the ALeRCE website. They often provide interactive Jupyter notebooks on GitHub.
  - **AMPEL Examples:** The ampel-contrib-sample repository is the best starting point.
  - **Lasair Guides:** The Lasair documentation has a "Documentation" section with step-by-step guides for creating your first filter.
  - **ANTARES Client Install:** Instructions are in the client's GitHub repository.
  - **Pitt-Google User Demos:** The repo explains how users can obtain credentials and start querying BigQuery or subscribing to Pub/Sub.

---

6. Practical Examples of Filters for Supernovae

Here is a summary of the examples seen, to get an immediate idea of the different approaches.

ALeRCE (Python with dedicated client):

```
python
# Search for objects classified as SN with probability > 0.8
from alerce_client import Alerce
client = Alerce()
results = client.query_objects(classifier='rf', class_name='SN',
probability=0.8)
```

Lasair (SQL):

```
sql
-- Select objects with classification 'SN' and g magnitude < 21
SELECT objectId, ra, dec FROM alerts
WHERE classification LIKE '%SN%' AND probability > 0.8 AND mag_g <
21;
```

ANTARES (Pure Python function):

```
python
# Define a filter as a Python function
def supernova_filter(alert):
    if alert["attributes"]["realbogus"] > 0.8 and\
alert["classifications"]["SN"] > 0.7:
        return True
    return False
```

Pitt-Google (SQL on BigQuery):

```
sql
-- Standard SQL query to extract bright candidates
SELECT objectId, ra, dec, magpsf, mjd
FROM `pitt_google_broker.alerts`
WHERE candidate.magpsf < 20.0 AND candidate.ndethist > 1;
```

Conclusion

The LSST era is not just a technological challenge but also an opportunity to rethink how we do science. Alert brokers are not mere filters; they become our primary digital collaborators. The choice of broker will depend on your scientific needs and technical skills:

- Want fast, automatic classification? **ALeRCE**.
- Need total algorithmic flexibility? **AMPEL**.
- Prefer the power of SQL? **Lasair** or **Pitt-Google**.
- Want an interactive, community-driven experience? **Fink**.

The advice is to explore the tutorials of the ones that seem most aligned with your interests.

Appendix

Interesting note from Community Forum by Roy Williams

Suppose you have an algorithm to decide if an LSST object is interesting enough to consider follow up observation with a big telescope. You start by splitting the algorithm into primary and secondary filters: first perhaps choosing only bright objects from a watchlist, with a host galaxy, or fast hot risers; then you have a code to analyse the lightcurve and other parameters in more detail, perhaps fitting a model or running a machine-learning algorithm.

Lasair is a platform for the primary filter: it offers Sherlock crossmatching with many catalogues, matching with your own uploaded watchlist, fitting an explosion model (BBB), colour temperature, and many other ways to filter. There is then near-real-time delivery via machine-readable Kafka.

Your secondary filter receives those alerts that pass that primary filter, that may include the light curve in six filters, then selects what is most interesting. There are a number of ways to handle those objects deemed interesting by the secondary filter.

- Annotate Lasair. Results from the secondary filter are bundled into a classification and a JSON packet to be sent back to Lasair 1. Others can now run their own web/API queries to discover these; there can be another filter for as those objects get new alerts. If you ask the Lasair team, we can make your annotator “fast”, meaning that as soon as an object is annotated, the filters that use it are run immediately and their Kafka streams updated.
- Automated message to a Slack channel. Your secondary filter posts messages automatically with a Slack webhook, then you and your group can see these and start discussion, with an eye to follow-up observations.
- Build a stream with Hopskotch or another Kafka-based streaming producer.

- Assemble results into a spreadsheet, webpage, or marshall system, for example xlsxwriter 1 or YSE-PZ 1, which can be reviewed by you and your group preceding followup.
- The secondary filter could initiate follow-up completely automatically, using TOM toolkit or another automated observation system.
- Convince the Lasair team that your algorithm should become part of their codebase, thus generating new “features” that will appear in the Lasair schema and therefore can be used to build a primary filter.
- Work with the Fink or Antares teams to add your filters and algorithm to the classification pipelines that they host, through creating a pull request.

Babamul & BOOM (link here)

Babamul is one of the seven "full-stream" brokers receiving and processing the full LSST alert stream. To access the alert search an account is needed, however, a*ccount creation will be enabled when LSST alerts are publicly available.*

Babamul is an open-source, lightweight, easily deployable broker that will initially deliver a low latency stream of Rubin alerts that match with ZTF objects. In the future, we aim to upgrade Babamul to transforms the massive and heterogenous LSST data stream into channels organized by astronomical event properties (e.g. fast, blue, ...) therefore dramatically reducing the data volumes. As such, astronomers can subscribe only to events relevant to their research and filter them further using minimal hardware and familiar tools (e.g. astropy, numpy, astroquery) while also allowing for fast and local ML-based workflows. Babamul also offers a number of open-sourced modules, ML models, example pipelines, and data management systems (with candidate vetting and follow-up UIs).

The unique strength of Babamul is its underlying architechture, or BOOM which allows for continuous crossmatch of ZTF and Rubin alert streams and the use of the combined lightcurves at filtering times.

BOOM stands for Bursts and Outbursts Observation Monitor and it refers to the underlying hardware and software infrastructure that enables a full blown science data platform for transient astronomy. BOOM features:

♦ A multi-survey data archive and alert broker (Babamul) combining data from the ZTF and Rubin alert stream to offer users combined lightcurve infomation at filtering times.
♦ An interactive collaborative marshal for the transient, variable, and solar system science cases
♦ Enriched data via multimodal ML models
♦ High throughput as BOOM is built from scratch with the Rust programming language,

with tier-like system of workers for maximum scalability. It relies on Valkey as an in-memory job queue, and on MongoDB as its database for it's built-in spatial querying features
♦ 100% customizable filters by the end user.

BOOM and Babamul are jointly developed by Caltech and our ZTF partners at UMN (Coughlin's group). The AI/ML capabilities of Babamul are enabled by AppleCiDEr. The source code and documentation are open source and hosted on Github.